The Return Type Rule and Generics

Karl Naden
Oracle Labs
karl.naden@oracle.com

Abstract

The combination of multiple inheritance and parametric polymorphism over types and functions adds technical complexity to a system with symmetric multiple dispatch on overloaded functions. In our OOPSLA 2011 paper, we proposed an interpretation of generic functions, overload rules, and type restrictions that allow us to write desirable overload sets. This write-up provides a more in-depth discussion of these rules as well as alternate options and justifies our choice.

1. Introduction

Dynamic dispatch, especially symmetric multiple dispatch, on overloaded functions, multiple inheritance, and parametric polymorphism are three powerful features of programming languages. Each has proven benefits in the literature. The Fortress programming language attempts to provide support for all three features. Our OOPSLA 2011 publication presented rules needed to allow modular checking of valid overload sets as well as type system features and restrictions that ensure that interesting overload sets are valid.

In particular, an interesting overload set that the paper discusses is the tail function:

$$tail[\![X]\!](x : \texttt{List}[\![X]\!]) : \texttt{List}[\![X]\!]$$
$$tail(x : \texttt{List}[\![\mathbb{Z}]\!]) : \texttt{List}[\![\mathbb{Z}]\!]$$

where a generic function that takes a generic type as an input is specialized for a particular instantiation of the type. This sort of function is desirable so that programmers can call functions that have the same effect by the same name. The only difference is the algorithm which can take advantage of a specific representation in the case of *def2*, so it seems natural to allow them to be called the same name, especially since the goal of the runtime is to choose the most specific applicable definition so that as much information can be used to make the algorithm efficient.

However, this overload is potentially unsound. Under the paper's interpretation of polymorphic functions, the second definition is considered to be strictly more specific than the first because it can accept a strictly smaller set of inputs. This means that the overloading passes both the no-duplicates and meet rules which restrict the domains of overloaded functions. However, the overloading is fundamentally unsound in the presence of multiple inheritance. If a type $\texttt{BadList} <: \{\texttt{List}[\![\texttt{String}]\!], \texttt{List}[\![\mathbb{Z}]\!]\}$ statically known to be a $\texttt{List}[\![\texttt{String}]\!]$ was passed to the *tail* function, then the statically guaranteed return type would be $\texttt{List}[\![\texttt{String}]\!]$. But at runtime the more specific function for $\texttt{List}[\![\mathbb{Z}]\!]$ would be chosen and return a $\texttt{List}[\![\mathbb{Z}]\!]$ which is not a subtype of $\texttt{List}[\![\texttt{String}]\!]$ thus breaking type safety.

Since these are overload sets that we want to express, the OOPSLA paper suggests adding a type-level restriction called *multiple instantiation exclusion* which outlaws `BadList` by preventing traits and objects from being subtypes of multiple instantiations of a single type. However, the paper does not further motivate this restriction, nor provide any alternative solutions, nor explain what overloads the restriction allows and why it works. We supplement the OOPSLA paper by expanding on this exposition to clearly explain why multiple instantiation exclusion works and is (arguably) the best choice for the Fortress programming language.

2. Terminology

We start with a brief introduction to the terminology of types and overloaded functions in the Fortress programming language. TODO - introduce types.

A function declaration consists of a name, a sequence of type parameters enclosed in white square brackets with associated bounds in curly braces, a type for the domain of the function, and the return type of the function. In the example

$$f[\![X <: \{M\}, Y <: \{N\}]\!](\texttt{List}[\![X]\!], \texttt{Tree}[\![Y]\!]) : \texttt{Map}[\![X, Y]\!]$$

f is the name of the function, X and Y are declared type parameters bounded by types M and N respectively, the tuple type $(\texttt{List}[\![X]\!], \texttt{Tree}[\![Y]\!])$ is the domain of the function, and $\texttt{Map}[\![X, Y]\!]$ is its return type. To reduce clutter, we omit the white square braces when there are no type parameters and the curly braces for singleton bounds.

A function declaration $d = f[\![\overline{X <: \{\overline{M}\}}]\!]S : T$ may be *instantiated* with type arguments $\overline{W}$ if $|\overline{W}| = |\overline{X}|$ and $W_i <: [\overline{W}/\overline{X}]N_{ij}$ for all i an j. We call $[\overline{W}/\overline{X}]f\ S : T$ an *instantiation* of d. When we do not need to refer to W, we can say $f(U) : V$ is an *instance* of d with the substitution implicit. We let $\mathcal{D}$ represent a finite collection of sets of function declarations and $\mathcal{D}_f$ the subset of $\mathcal{D}$ that contains all declarations with name f.

An instance $f(U) : V$ of a declaration d is *applicable* to a type T if and only if $T <: U$. A function declaration is applicable to a type if and only if at least one of its instances is. For any type T, the set $\mathcal{D}_f(T)$ contains precisely those declarations in $\mathcal{D}_f$ that are applicable to T. We define a specificity relationship $\preceq$ on function declarations. Intuitively, one declaration is more specific than another if it is applicable to a strictly smaller set of types. It is implemented as existential subtyping on the domain of the declarations.

For overload sets to be valid, there cannot be any ambiguity in which function declaration should be used for a given type. To achieve this, we require that for a given overload set $\mathcal{D}_f$ and type

T there exists a unique definition $d \in \mathcal{D}_f(T)$ such that for all $d' \in \mathcal{D}_f(T)$ we have $d \preceq d'$. We call this definition the *most specific applicable definition* or $\mathbf{MSAD}(\mathcal{D}_f(T))$. This requirement is enforced by the *No Duplicates Rule* which ensures that no two definitions are equally specific and the *Meet Rule* which checks that the overload set forms a meet semi-lattice under the specificity relationship. A third rule, the *Return Type Rule* makes sure that executing the most specific applicable definition at runtime is always type safe. This last rule is the subject of the next section.

3. Ensuring Type Safety

When choosing a function definition to execute at runtime, it is not enough to simply choose the most specific definition. The runtime must also preserve type safety by choosing a function definition that returns a subtype of the return type guaranteed by the static type system. At compile time, Fortress' type system finds the **MSAD** for each function call based on the static information it has available. This provides a static return type. The runtime may have more information about the actual types of the inputs that allows it to select a more specific function definition based on the parameter types. However in order to preserve type safety, it can only choose a definition that returns a subtype of the return type given by the static **MSAD**. There are two approaches to this requirement

1. Automatically restrict the choices the runtime can make within an overload set, or
2. Restrict overload sets to force programmers to write overloads that only offer sound choices at runtime.

The second restricts what overload sets are expressible while the first prevents us from always choosing the most specific function based on the types at runtime. We use examples to help us flesh out these two approaches and their associated tradeoffs.

3.1 Overloading Examples

It is not hard to write a function overload that could be unsound if the most specific function definition is chosen at runtime. Let $\mathcal{D}_{confused} =$

$$confused(\texttt{Object}) : \texttt{String}\ (*)\mathit{def1}$$
$$confused(\mathbb{Z}) : \mathbb{Z}\ (*)\mathit{def2}$$

To see why this is potentially unsound, consider an application of *confused* to an `Object` at compile time. $\mathcal{D}_{confused}(\texttt{Object})$ includes only *def1* and so this is the **MSAD** and the return type is `String`. However, it is possible at runtime to discover that the parameter is in fact an object of type $\mathbb{Z}$. $\mathcal{D}_{confused}(\mathbb{Z})$ includes both *def1* and *def2* and since *def2* $\preceq$ *def1*, *def2* would be the **MSAD**. However, because the return type $\mathbb{Z}$ of *def2* is not a subtype of `String`, this function definition could not be chosen without breaking type safety.

It is not clear that we should support the *confused* overload. The purpose of an overload set it to be able to determine which definition should be executed dynamically. However, the restriction on the return type means that whichever definition is chosen statically is the one that must be executed dynamically to preserve type safety. This suggests that the programmer should be forced to give each definition separate names and explicitly choose the correct definition.

The same intuition does not apply when considering overload sets that include polymorphic functions. Consider the polymorphic identity function and a specialization for a particular type. $\mathcal{D}_{polyBad} =$

$$polyBad[\![X]\!](X) : X\ (*)\mathit{def1}$$
$$polyBad(\mathbb{Z}) : \mathbb{Z}\ (*)\mathit{def2}$$

It seems logical that we might want to have special functionality for a particular representation. However, if *def2* was not visible at compile time but was available at runtime (as is possible in the Fortress component system), then this overload set could be unsound. Consider an argument of type $\mathbb{N}$. *def1* provides a return type of $\mathbb{N}$, but the more specific *def2* found at runtime will provide a $\mathbb{Z}$ if it is allowed to be chosen. We can fix this issue by making *def2* polymorphic as well: $\mathcal{D}_{polyOk} =$

$$polyOk[\![X]\!](X) : X\ (*)\mathit{def1}$$
$$polyOk[\![Y <: \mathbb{Z}]\!](Y) : Y\ (*)\mathit{def2}$$

Now, in the above situation *def2* is simply instantiated at $\mathbb{N} <: \mathbb{Z}$ and type safety is maintained.

We are not so lucky when considering examples that involve generic types such as the *tail* example from the introduction. Consider a similar definition for the *head* function: $\mathcal{D}_{head} =$

$$head[\![X]\!](\texttt{List}[\![X]\!]) : X\ (*)\mathit{def1}$$
$$head(\texttt{List}[\![\texttt{Bit}]\!]) : \texttt{Bit}\ (*)\mathit{def2}$$

To see the potential for unsoundness, consider the type `ConfusedList` $<:$ $\{\texttt{List}[\![\texttt{Bit}]\!], \texttt{List}[\![\texttt{String}]\!]\}$. `ConfusedList` could be known statically as a $\texttt{List}[\![\texttt{String}]\!]$ and the $\mathbf{MSAD}(\mathcal{D}_{head}(\texttt{List}[\![\texttt{String}]\!])$ is *def1* with a return type of `String`. At runtime, $\mathbf{MSAD}(\mathcal{D}_{head}(\texttt{ConfusedList})$ is *def2* which returns a `Bit` which is not a subtype of `String`. Furthermore, the trick of making the second definition polymorphic does not work as there is no way to return a combination of the elements from each list. [1] Therefore, if we want the flexibility to write overloads that manipulate generic structures but still allow special cases, we need some way to allow these overload sets.

4. Limiting Runtime Choices Automatically

In order to allow overloads like *tail* and *head*, one option is to prevent the runtime from choosing function definitions that cause the unsoundness. One obvious place to put this logic is when determining function applicability at runtime.

4.1 Dynamic Dispatch

The runtime is already responsible for inspecting the runtime types of the inputs to a function call and selecting the most specific applicable function definition to execute. It does so by solving a set of constraints based on the input types and the parameter types of the different definitions in the overload. The runtime could be enhanced to include return type constraints when considering applicability.

For example, in the *head* example above we know statically that `String` must be returned from the call to *head*. Therefore, when the runtime checks if *def2* is applicable, it checks that it can return a subtype to `String`. Since it cannot, this definition is not chosen, even if it is applicable to the provided arguments.

4.2 Static Transformation

We notice that we actually could tell that *def2* could not be used at compile time once we found that the return type needed to be a `String`. This suggests that we could limit the choices that the runtime can make by rewriting overload sets and function calls.

In the case of a call to *head* with a parameter with static type $\texttt{List}[\![\texttt{String}]\!]$, the compiler would recognize that *def2* could not apply at runtime and partition the overload set. In fact, this particular set could be partitioned easily into two functions

[1] We can add a third definition to disambiguate, but the reasoning is complex: $head[\![Y, Z <: \{\texttt{List}[\![Y]\!], \texttt{List}[\![\texttt{Bi}]\!]\}]\!](Z) : Y$. Essentially, we force the original return type to be used by defining a yet more specific definition that must apply if both do. Unfortunately, besides being complex, it is non-local.

$head\_poly[\![X]\!](\texttt{List}[\![X]\!]) : X$
$head\_bit(\texttt{List}[\![\texttt{Bit}]\!]) : \texttt{Bit}$

At compile time, we know that any application of *head* to a subtype of `List[[Bit]]` uses *def2* and everything else uses *def1*. Therefore, the compiler could rewrite these calls into calls to *head_bit* and *head_poly* respectively.

In general, overload sets would be partitioned to prevent type information known at compile time from being discarded to choose a more specific function definition at runtime. Though complex in some cases, we believe that this translation would be possible (though it might not interact well with modules and hiding).

4.3 Evaluation

These solutions achieve the goal of allowing names like *head* and *tail* to be overloaded in the manner that we want. However, they do so at the price of complicating the concept of an overload set. Each call site of a given function name has a different overload set in the sense that the static context may restrict what functions are applicable. One implication of this is that programmers can no longer understand the execution of a program without static type information. By adding contextual information to the concept of the overload set, we muddy its semantics in an undesirable way.

Secondly, this rule allows non-sensical overloads such as *confused* above. We can justify our desire to write examples like *head* and *tail*. In contrast, *confused* does not seem to have much use, further diluting the usefulness of overload sets as a concept.

Another positive aspect of these solutions is that they recognize a quirk of the function specificity relationship. If statically *def1* of *head* applies instantiated at `String` with a domain of `List[[String]]`, is it really the case that *def2* with a domain of `List[[Bit]]` is more specific? The rules would say no in these monomorphic cases. This suggests that by checking specificity at the polymorphic level, we are causing monomorphic instantiations to have a specificity relationship they would not otherwise have. However, our previous work showed that this version of specificity is useful for other reasons, so changing it is not an option.

Given this high price in added complexity, we do not consider these to be good options and so turn to restrictions on overload sets.

5. Manual Restrictions

In order to ensure type safety without muddying the semantics of overloading we propose restrictions to overload sets. However, since we also want to be able to write overloads like *tail* and *head*, we also propose restrictions on the type system that allow this kind of overload.

5.1 The Return Type Rule

Examples like *head* and *tail* are unsound because a more specific function definition can be chosen at runtime that returns a type that is not more specific. The return type rule prevents this:

> Given a pair of function definitions $d_2 \preceq d_1$ in an overload set and a type T to which both are applicable. For every instance of d_1 applicable to T with return type R_1, there exists an instance of d_2 that is applicable to T which has a return type $R_2 <: R_1$.

The return type rule guarantees that if a more specific function definition is applicable, then the static return type cannot prevent it from being used because there must exist an instance that is both applicable and provides a type safe return type.

For example, taking the *head* example where *def1* is d_1 and *def2* is d_2 and $T = \texttt{ConfusedList}$. *def1* instantiated at `String` is applicable to T with return type `String`. However, *def2* can only return a `Bit` which is not a subtype of `String` and thus the *head* overload fails the return type rule. By similar reasoning, examples like *confused* above also fail this rule.

5.2 Restricting the Type System

Unfortunately, the return type rulerejects overload sets such as *head* and *tail* that we are interested in writing. Since we saw that weakening this rule led to confusing semantics, we look for other ways to allow these overload sets. One potential way to do this is by preventing the types that cause problems. Our problematic examples suggest that types that extend multiple instantiations of a single type are the culprits. We restrict these types with a new kind of exclusion.

5.2.1 multiple instantiation exclusion

The goal of *multiple instantiation exclusion* is to make these types illegal. Multiple instantiation exclusion says that any type which is a subtype of two distinct instantiations of a single generic type is equivalent to `Bottom`, the uninhabited type. This means that examples such as `ConfusedList` are no longer problematic because they are not valid types.

Multiple instantiation exclusion does restrict expressibility of the type system by preventing programmers from writing types that they might want to write such as $\texttt{HasStringAndInt} <: \{\texttt{Has}[\![\texttt{String}]\!], \texttt{Has}[\![\mathbb{Z}]\!]\}$ for example. However, since it removes the examples that cause the overloads *head* and *tail* to fail the return type rule, we hypothesize that it will make them safe. In order to prove this, we first need to characterize the types of overload sets that we want to allow.

5.2.2 More Examples

We already saw that *head* and *tail* are examples of overloads that we want to write that are potentially unsound. We believe (and will shortly prove) that multiple instantiation exclusion makes these overloadings safe by eliminating problematic examples. However, in order to prove that it works, we need to characterize the overload sets that we are interested in allowing. The general form of these overload sets is that they contain a polymorphic function for manipulating the general form of a polymorphic type along with a function specialized to take advantage of a specific representation for a particular instantiation of that type. Both *head* and *tail* take a generic type as an input and output either the same type or its instantiation. Given our *polyOk* example that is not unsound, the generic type appears to be important. But does it need to be an input, and does the return type need to be related?

The *size* function addresses the second question. Let $\mathcal{D}_{size} =$

$size[\![X]\!](\texttt{List}[\![X]\!]) : \mathbb{Z}\ (*)\ def1$
$size(\texttt{List}[\![\texttt{Bit}]\!]) : \mathbb{Z}\ (*)\ def2$

Since the return type is the same for both definitions, the return type rule does not fail. Therefore, it seems that there does need to be a relationship between the instantiation of the input type and the return type.

For the first question, we consider a factory method for singleton lists. Let $\mathcal{D}_{singletonList} =$

$singletonList[\![X]\!](X) : \texttt{List}[\![X]\!]\ (*)\ def1$
$singletonList(\texttt{Bit}) : \texttt{List}[\![\texttt{Bit}]\!]\ (*)\ def2$

This example is unsound even with multiple instantiation exclusion. The $\mathbf{MSAD}(\mathcal{D}_{singletonList}(\texttt{Object})$ is *def1*, but at runtime we could get a $\texttt{Bit} <: \texttt{Object}$ which would make choosing the more specific *def2* unsound. Since our example was not of the same form as `ConfusedList` multiple instantiation exclusion will not help. On closer inspection, this does not appear to be as problematic because intuitively this sort of overload should not be valid. This function creates a `List` that starts with a single element, but to

which other elements could later be added. Given that the type of the first element added to the list does not necessarily represent the type of all the elements added we actually do not want the runtime system to choose a more specific instantiation for a call to *singletonList* at runtime as it may cause later uses of the list to become unsound. Therefore, it is appropriate that this overload is rejected even with the added exclusion.

Finally, we note that the more specific definition does not need to be monomorphic. For instance, if we wanted to specialize some function for all lists of numbers, we would write an overload set like $\mathcal{D}_{specializeRange} =$

$$specializeRange[\![X]\!](\texttt{List}[\![X]\!]) : X\ (*)\, def1$$
$$specializeRange[\![Y <: \texttt{Number}]\!](\texttt{List}[\![Y]\!]) : Y\ (*)\, def2$$

This sort of overloads acts much like the monomorphic examples and appears to be allowed with multiple instantiation exclusion.

These examples give us confidence that we can characterize the type of overloads that we are trying to enable. Informally, these are overloaded functions that take generic types as arguments and return a type related to that generic type. Formally,

Definition 5.1 (special case)**.** *Given a generic function definition $f[\![X <: U]\!](x : T[\![X]\!]) : R$ where T is a generic trait and X appears in R. We say that for any $V <: U$ the following definitions constitute special cases*

1. *$f[\![Y <: V]\!](x : W) : R'$ where Y appears in W and R' and for all $V' <: V$ $[V'/Y]W <: T[\![V']\!]$ and $[V'/Y]R' <: [V'/X]R$*
2. *$f(x : W) : R'$ where $W <: T[\![V]\!]$ and $R' <: [V/X]R$*

This definition is not as general as it could be as in reality, we would want to account for other parameters. However, it captures the key concepts in a relatively simple definition. We note that *tail*, *head*, and *specializeRange* are all overloaded functions which include a generic function definition and a special case.

5.2.3 Proof

We can now prove that multiple instantiation exclusion allows these special cases to pass the return type rule.

Lemma 5.2. *Given a generic function definition $d = f[\![X <: U]\!](x : T[\![X]\!]) : R$. If d' is a special case of d using $V <: U$, then d' is more specific than d and furthermore, d and d' pass the return type rule.*

Proof. The definition of a special case is sufficient to prove that d' is more specific than d. To show the return type rule succeeds, we take an ilk I to which both d and d' are applicable. Since d' is applicable to I, there exists some $A <: V$ such that $I <: T[\![A]\!]$. By multiple instantiation exclusion, we know that there exists no $B \neq A$ such that $I <: T[\![B]\!]$. Since generic types are invariant, we know that there exist no $B \neq A$ such that $T[\![A]\!] <: T[\![B]\!]$. Therefore, A is the only instantiation of X for which d is applicable to I. The definition of a special case guarantees that this pair of instantiations pass the return type rule and since it is the only pair of instantiations that are applicable, we are done. □

The proof is informative in terms of how this restriction achieves the desired effect. Effectively, multiple instantiation exclusion rules out any instantiation other than the one known at compile time, thereby preventing the runtime from having the option of choosing a different instantiation. We can therefore trust the instantiation known at compile time.

5.3 Evaluation

There are several benefits of this scheme over the prior proposals. First, overload sets do not depend on the calling context, but can be understood as a separate concept. Secondly, multiple instantiation exclusion provides a simple restriction to the type system that provably allows overrides that we want to write (*tail*) while still prohibiting questionable overloads (*confusing*).

The price that we must pay is the restriction on the type system. This means that we cannot use some types as we might like. For instance, given a type $\texttt{Has}[\![X]\!]$, we cannot define $\texttt{hasStringAndInt} <: \{\texttt{Has}[\![String]\!], \texttt{Has}[\![\mathbb{Z}]\!]\}$. On the other hand, it is unclear how to use these kind of types and in particular whether you could reasonably write overload sets that are specialized. In other words, you would need to use the static information rather than dynamic dispatch. This suggests that perhaps we could consider adding syntax to allow some generic traits to be be multiple instantiation excluded and others not. The tradeoff would be specialization of generic functions or multiple instantiation inheritance, but not both.

On the other hand, when we consider the runtime system, we realize that there is a strong benefit to multiple instantiation exclusion. It guarantees that for any type that extends a generic trait, there exists a canonical instantiation of that trait for that object. This is important for overloads such as

$$headOrID(\texttt{Object}) : \texttt{Object}\ (*)\, def1$$
$$headOrID[\![X]\!](\texttt{List}[\![X]\!]) : X\ (*)\, def2$$

If *def1* is all that is known at compile time, but at runtime the object is a `ConfusedList`, which instantiation of *def2* is used, `Bit` or `String`? Context does not give us any clue because we only need an `Object`. Thus, we need to choose an instantiation without any means to do so. Outlawing this type of overload is a slippery slope into special cases.

Ultimately, Fortress implements blanket multiple instantiation exclusion for the semantic benefits and the simplicity of the restriction.

6. Variant Generics

In the above sections, we assumed that generics were *invariant* in the sense that $T[\![X]\!] <: T[\![Y]\!]$ if and only if $X = Y$. However, many abstractions can support more permissive subtyping between generic instantiations. Fortress supports variant generic types, including

1. **Covariance**: $T[\![X]\!] <: T[\![Y]\!]$ if and only if $X <: Y$
2. **Contravariance**: $T[\![X]\!] <: T[\![Y]\!]$ if and only if $Y <: X$

We deal explicitly with covariance only as for our purposes, contravariance is the same with the subtyping relationship switched.

6.1 Covariant multiple instantiation exclusion

We need to update the definition of multiple instantiation exclusion to include covariant generic types. We could leave the restriction the same and prevent multiple instantiations just like for invariant generics. However, this is more restrictive than we need. In fact, as long as we can identify some minimal instance we can ensure type safety.

To see why, consider the *head* function on the covariant list type $\texttt{CoList}[\![X]\!]$. Additionally, we specialize it for some type `A`.

$$coHead[\![X]\!](\texttt{CoList}[\![X]\!]) : X\ (*)\, def1$$
$$coHead[\![Y <: \texttt{A}]\!](\texttt{CoList}[\![Y]\!]) : Y\ (*)\, def2$$

Note that we must make *def2* generic. This is for the same reasons as the *polyOk* example above. Consider the type `B` unrelated to but not excluding `A` and $\texttt{C} <: \{\texttt{A}, \texttt{B}\}$. The type $\texttt{AmbiguousCoList} <: \{\texttt{CoList}[\![A]\!], \texttt{CoList}[\![B]\!]\}$ causes the *coHead* overload to fail the return type rule. Type parameter X of *def1* can be instantiated to `B` and since `B` is not a subtype of `A`, there is no way to instantiate Y of *def2* to be both applicable

and type safe. However, if we add $\texttt{CoList}[\![C]\!]$ to make the type $\texttt{OkCoList} <: \{\texttt{CoList}[\![A]\!], \texttt{CoList}[\![B]\!]\texttt{CoList}[\![C]\!]\}$, then the problem goes away since we can now choose to instantiate *def2* at `C` and return $\texttt{C} <: \texttt{B}$ which is sound.

Thus, to multiple instantiation exclusion, we add that if the type parameter is covariant, then a type can extend multiple instantiations of that type parameter as long as there exists a minimal instantiation that is a subtype of all other instantiations. This is the ancestor meet rule that appears in the failed OOPSLA 2012 submission.

6.1.1 Cross Instantiation

There is one more problematic situation that we did not encounter with invariant type parameters. Consider the type $\texttt{CoArrayList}[\![X]\!] <: \texttt{CoList}[\![X]\!]$ which holds for all X. Now, we specialize *coTail* for the array list case:

$$coTail[\![X]\!](\texttt{CoList}[\![X]\!]) : \texttt{CoList}[\![X]\!]\ (*)\,\textit{def1}$$
$$coTail[\![Y]\!](\texttt{CoArrayList}[\![Y]\!]) : \texttt{CoArrayList}[\![Y]\!]\ (*)\,\textit{def2}$$

We consider *def2* to be more specific as usual since it handles a subset of the inputs of *def1*. Now, consider the type $\texttt{CrossInstance} <: \{\texttt{CoArrayList}[\![\texttt{Object}]\!], \texttt{CoList}[\![\texttt{Number}]\!]\}$. This type provides a counter example to the return type rule for the *coTail* overload. We can instantiate *def1* to `Number`, but in order to make *def2* applicable, we must instantiate it at `Object` and `Object` is a not a subtype of `Number`.

In light of this problem, we must add a second restriction to prevent these crossed instantiations.

> Given type $\texttt{T}[\![X]\!]$ where X is covariant and type $\texttt{S}[\![Y]\!]$ were Y is covariant and for all Z $\texttt{S}[\![Z]\!] <: \texttt{T}[\![Z]\!]$. Let `A` and `B` be distinct ground types where $\texttt{A} <: \texttt{B}$. If $I <: \texttt{S}[\![B]\!]$ and $I <: \texttt{T}[\![A]\!]$, then $I <: \texttt{S}[\![A]\!]$.

With these two restrictions, we can prove that special case overloads involving covariant type parameters will now pass the return type rule as well (TODO).

6.1.2 Exception for selfless types

This restriction is problematic for expressing certain types of relationships such as the numeric hierarchy that is expressed using the so called self-type idiom (now selfless types or trolls). Some sets of numbers and the operators over them give rise to special algebraic properties known as rings and the more specialized fields. Fortress aims to support them in the following way:

$$\textbf{trait}\ \texttt{Ring}[\![X <: \texttt{Number}]\!]\ \textbf{comprises}\ X\ \ldots\ \textbf{end}$$
$$\textbf{trait}\ \texttt{Field}[\![X <: \texttt{Number}]\!]\ \textbf{comprises}\ X\ \textbf{extends}\ \texttt{Ring}[\![X]\!]\ \ldots\ \textbf{end}$$

The self-type idiom is realized with the **comprises** clause which contains the instantiation of the self type. The **comprises** clause limits the types that can extend the given type to those appearing in the clause. In this idiomatic usage, the type $\texttt{Ring}[\![X]\!]$ is identified as X. Thus a $\texttt{Field}[\![\mathbb{Q}]\!]$ is really just $\mathbb{Q}$ itself.

Unfortunately, the numerical hierarchy contains crossed instantiations of the kind that we determined must be outlawed:

$$\textbf{trait}\ \mathbb{Q}\ \textbf{extends}\ \texttt{Field}[\![\mathbb{Q}]\!]\ \ldots\ \textbf{end}$$
$$\textbf{trait}\ \mathbb{Z}\ \textbf{extends}\ \{\mathbb{Q}, \texttt{Ring}[\![\mathbb{Z}]\!]\}\ \ldots\ \textbf{end}$$

A $\mathbb{Z}$ is a $\texttt{Field}[\![\mathbb{Q}]\!]$ and a $\texttt{Ring}[\![\mathbb{Z}]\!]$ but not a $\texttt{Field}[\![\mathbb{Z}]\!]$ which breaks our rule given above. As above, we can come up with an example that is unsound if we allow this type hierarchy.

$$add[\![X]\!](\texttt{Ring}[\![X]\!], \texttt{Ring}[\![X]\!]) : \texttt{Ring}[\![X]\!]\ (*)\,\textit{def1}$$
$$add[\![Y]\!](\texttt{Field}[\![Y]\!], \texttt{Field}[\![Y]\!]) : \texttt{Field}[\![Y]\!]\ (*)\,\textit{def2}$$

Take type $\mathbb{Z}$ which is provided as the inputs to *add*. Both *def1* and *def2* are applicable. But if we instantiate *def1* at $\mathbb{Z}$, then we have a return type of $\texttt{Ring}[\![\mathbb{Z}]\!]$ for the less specific definition. The more specific *def2* is only applicable when instantiated at $\mathbb{Q}$, but the resulting return type $\texttt{Field}[\![\mathbb{Q}]\!]$ is not a subtype of $\texttt{Ring}[\![\mathbb{Z}]\!]$, thus witnessing the failure of the return type rule.

Recall that the return type rule handles two failure cases: first, the possibility that more specific type information at runtime will cause a more specific function definition to be chosen. Second is that the more specific function may not be visible to the compiler (hidden behind an API). Note that the first failure case should not actually apply here. When we statically know that we have a $\texttt{Ring}[\![\mathbb{Z}]\!]$, we actually know much more than this. The self-type idiom means that this type is equivalent to $\mathbb{Z}$. But $\mathbb{Z} <: \mathbb{Q} <: \texttt{Field}[\![\mathbb{Q}]\!]$ which implies that based on static information *def2* should apply. However, this does not extend to the second failure case when the more specific definition is hidden.

One way to ensure that the first situation cannot happen is to rewrite the overload so that the self-type is a bound rather than the parameter type:

$$add[\![X <: \texttt{Ring}[\![X]\!]]\!](X, X) : X\ (*)\,\textit{def1}$$
$$add[\![Y <: \texttt{Field}[\![Y]\!]]\!](Y, Y) : Y\ (*)\,\textit{def2}$$

This forces us to recognize the type $\texttt{Ring}[\![\mathbb{Z}]\!]$ as $\mathbb{Z}$ in order for *def1* to apply which will automatically also mean that *def2* applies. This approach suggests further that the self-type idiom should be more than simply an idiom and instead a separate construct which cannot be used as a first class type, but instead only in bounds. Victor and John have started working on this sort of theory with their *trolls*. However, the visibility problem shows that this is not sufficient to fix the unsoundness problems with the cross instantiation. Therefore, more work needs to be done in order to understand how to best support the numerical hierarchy in the Fortress type system.